CAL POLY POMONA

MAT 3810

Operations Research

Integer Programming Problems

Authors
Jeremy Lin
Thuy Lu

Academic Mentor
Dr. Jillian Cannons

December 10, 2020

Contents

Introduction **2**

Base Problem **2**
Problem Description 2
Mathematical Formulation 3
Result and Discussion 5

Extension Problem **6**
Problem Description 6
Mathematical Formulation 6
Result and Discussion 7

Conclusion **8**

Appendix **9**
Base Case Code 9
Extension Case Code 28

References **51**

Introduction

In this project, we formulated an integer programming problem to solve an assignment problem where a number of health care workers needed distributing to 5 hospitals in response to a medical crisis situation. The workers available for assignments are Nurse Practitioners, Specialists of Type A, B, C and D. We included a base case and an extension case, in which we considered different numbers of health care workers each hospital required and assigned the workers accordingly in order to maximize benefits for the hospitals.

Base Problem

Problem Description

There available limited numbers of Nurse Practitioners, Specialists of Type A, B, C, and D to be assigned to the hospitals. The total number of each is given in Table 1. If a hospital is assigned a certain number of workers of a specific type, they receive a benefit as tabulated in Table 2.

	Specialist Type				Nurse Practitioner
	A	B	C	D	
Count	8	3	4	3	250

Table 1: Number of workers available for assignment to hospitals

Count	Benefit for Indicated Count for Specialist Type				Nurse Practitioner
n	A/$f_A(n)$	B/$f_B(n)$	C/$f_C(n)$	D/$f_D(n)$	
0	0	0	0	0	0
1	0	1	0	2	1
2	7	2	1	4	2
3	11	5	3	9	3
4	14	-	5	-	4
5	16	-	-	-	5
6	18	-	-	-	6
7	19	-	-	-	7
8	21	-	-	-	8
n	-	-	-	-	n

Table 2: Benefits of assigning differing numbers of workers to hospital

There are number of requirements which must be met when assigning the workers to the hospitals. In particular, each hospital must be assigned

- at least 20 Nurse Practitioners,
- at least one Specialist of Type A
- at least one Specialist of Type B or C, and
- at most one Specialist of Type D.

Mathematical Formulation

Let the followings be the decision variables:

$$y_{Hij} = \begin{cases} 1 & j \text{ numbers of Specialist of Type H is assigned to hospital } i \\ 0 & j \text{ numbers of Specialist of Type H is not assigned to hospital } i \end{cases} \qquad \forall H = A, B, C$$

$$y_{Di} = \begin{cases} 1 & \text{Specialist of Type D is assigned to hospital } i \\ 0 & \text{Specialist of Type D is not assigned to hospital } i \end{cases}$$

$$x_{Ni} = \text{numbers of Nurse Practitioners assigned to hospital } i$$

According to the problem description, we would like to find out how many Specialists and Nurse Practitioners can be assigned to each hospital. We defined y_{Hij} to allow us to track the number of specialists assigned to a hospital and map it directly to the corresponding benefit, as we will see in more details below. Also, due to the constraint at most one Specialist of Type D can be at each hospital, hospital i only requires one variable y_{Di} to demonstrate whether any Specialist of Type D is assigned to it. As for Nurse Practitioners, since the benefits of assigning n of them equals to the value n itself, we specified x_{Ni} to be the numbers of Nurse Practitioners assigned to a hospital as well as the benefit obtained from assigning that many Nurse Practitioners to that hospital.

A. Objective

The goal of this problem is to maximize benefits for the five hospitals that are each assigned certain numbers of Specialists of different types and Nurse Practitioners.

- First, we let $f_H(j)$ be a function that represents the benefit of assigning j Specialist of Type H. The values of $f_H(j)$ can be read from Table 2, where H = A, B, C, and D. For example $f_A(3) = 11$ means when 3 Specialists of Type A are assigned to a hospital, the benefit is 11.
- Furthermore, in the case of Specialists of Type A, B, and C, since the decision variable y_{Hij} indicates how many j Specialists of Type H are assigned to a hospital, we multiply this variable by the benefit $f_H(j)$ accordingly. Summing all those terms up over 5 hospitals gives a total benefits for allocating the three types of Specialists, with j corresponds to the available number of Specialists of each type.
- For Specialist of Type D, as mentioned before, a maximum of one can be assigned to a hospital. Then, the benefit relative to y_{Di} is $f_D(1) = 2$ and the benefit coming from assigning Specialists of Type D is the sum of $2y_{Di}$ for $i = 1, 2, \cdots, 5$. Lastly, the decision variable x_{Ni} has integer values and equals its benefit. Thus, summing x_{Ni}'s up over 5 hospitals would result in a total benefits from allocating Nurse Practitioners.

The objective function is written as follow:

$$Z = \sum_{j=1}^{8} \left(f_A(j) \sum_{i=1}^{5} y_{Aij} \right) + \sum_{j=1}^{3} \left(f_B(j) \sum_{i=1}^{5} y_{Bij} \right) + \sum_{j=1}^{4} \left(f_C(j) \sum_{i=1}^{5} y_{Cij} \right) + \sum_{i=1}^{5} 2y_{Di} + \sum_{i=1}^{5} x_{Ni}$$

B. Constraints

Constraint (1) ensures the mutual exclusivity of y_{Aij} variable for each hospital i, and at least one Specialist of Type A is assigned to each hospital:

$$\sum_{j=1}^{8} y_{Aij} = 1, \qquad \forall i = 1, 2, \cdots, 5 \tag{1}$$

Constraint (2) ensures that 8 Specialists of Type A are assigned to five hospitals:

$$\sum_{i=1}^{5} \sum_{j=1}^{8} j \cdot y_{Aij} = 8 \tag{2}$$

Constraint (3) ensures that 3 Specialists of Type B are assigned to five hospitals:

$$\sum_{i=1}^{5} \sum_{j=1}^{3} j \cdot y_{Bij} = 3 \tag{3}$$

Constraint (4) ensures 4 Specialists of Type C are assigned to five hospitals is 4:

$$\sum_{i=1}^{5} \sum_{j=1}^{4} j \cdot y_{Cij} = 4 \tag{4}$$

Constraint (5) ensures the mutual exclusivity of y_{Bij} for each hospital i:

$$\sum_{j=1}^{3} y_{Bij} \leq 1, \qquad \forall i = 1, 2, \cdots, 5 \tag{5}$$

Constraint (6) ensures the mutual exclusivity of y_{Cij} for each hospital i:

$$\sum_{j=1}^{4} y_{Cij} \leq 1, \qquad \forall i = 1, 2, \cdots, 5 \tag{6}$$

Constraint (7) ensures there is at least one Specialist of Type B or C assigned to hospital i:

$$\sum_{j=1}^{3} j \cdot y_{Bij} + \sum_{j=1}^{4} j \cdot y_{Cij} \geq 1, \qquad \forall i = 1, 2, \cdots, 5 \tag{7}$$

Constraint (8) ensures there is at most 1 Specialist of Type D assigned to a hospital:

$$x_{Di} \leq 1, \qquad \forall i = 1, 2, \cdots, 5 \tag{8}$$

Constraint (9) ensures that 3 Specialists of type D are assigned to the hospitals:

$$\sum_{i=1}^{5} x_{Di} = 3 \tag{9}$$

Constraint (10) ensures there are at least 20 nurse practitioners assigned to each hospital:

$$x_{Ni} \geq 20, \qquad \forall i = 1, 2, \cdots, 5 \tag{10}$$

Constraint (11) ensures that 250 Nurse Practitioners are assigned to the five hospitals:

$$\sum_{i=1}^{5} x_{Ni} = 250 \tag{11}$$

Result and Discussion

Solving the integer programming problem formulated using `MATLAB`, we found the assignments of health care workers as listed in Table 3 with a maximized benefit of $Z = 283$. Our group discovered that the solution of our Integer LP problem is not unique, which means that there are multiple solutions that give us a benefit of 283. We found at least two solutions as such.

Worker / Hospital	Specialist Type				Nurse Practitioner
	A	B	C	D	
1	2	1	0	1	170
2	2	0	1	1	20
3	2	1	0	1	20
4	1	1	0	0	20
5	1	0	3	0	20

Table 3: The allocations of health care workers to the hospitals

We also noticed that the distribution of the benefits, especially benefits obtained from the Nurse Practitioners is not even among the hospitals, as can be seen from the total benefits of each hospital from Table 4. This might have been the case since the hospitals does not differ by any characteristics. The benefits they receive from being assigned a certain number of workers are identical, thus it does not matter where the Nurse Practitioners are allocated.

Benefits Obtained by Hospital					Total
1	2	3	4	5	Benefits
180	29	30	21	23	283

Table 4: The maximized benefits of each hospital

To address the problem, we considered further to apply an upper bound on the number of Nurse Practitioners allocated each hospital such that the distribution is even among them. It turns out in this case 50 Nurses Practitioners are allocated to each hospital, resulting in the same maximized benefit of $Z = 283$. The allocation of the cases are shown in Figure 1. The bar graph on the left shows the distribution of worker without adding the upper bound constraint. The bar on hospital 1 is significantly higher than the rest of the hospitals, which means most of our resources is assigned to hospital 1. The bar graph on the right shows the workers distribution after adding the upper bound of 50 on the Nurse Practitioners variable. Each bar have similar height, which implied that each hospital have been assigned similar number of workers.

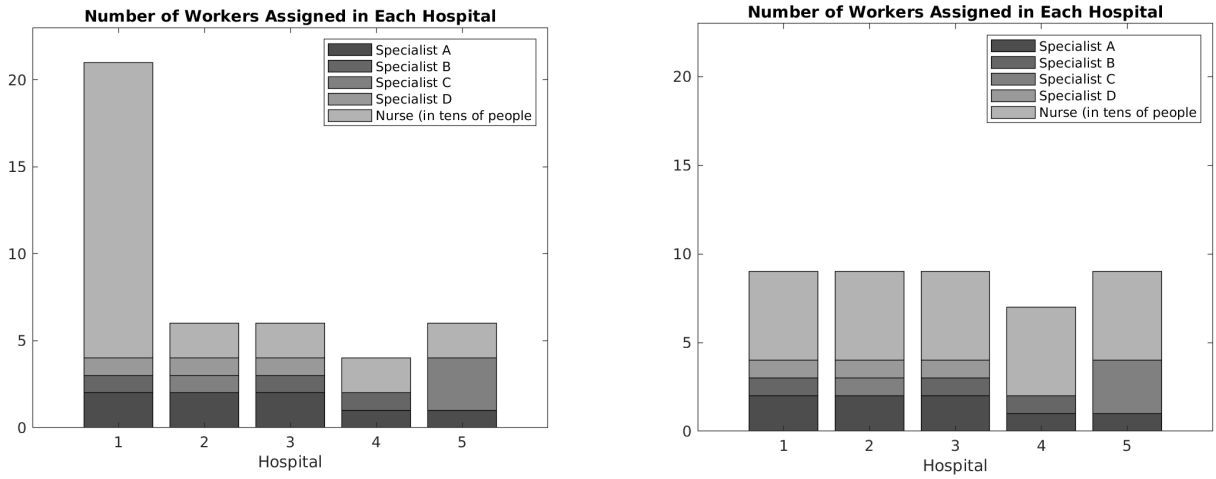

Figure 1: Allocation of workers for each hospital (nurses are counted in tens of people). Left: Without adding constraint to balance nurse allocation. Right: Adding an upper bound constraint to balance the allocation of Nurse Practitioners

Extension Problem

Problem Description

In the extension case, we considered Specialists of Type B can be assigned to more than one hospitals. However, for each person assigned to more than one hospital, there is a fixed decrease in benefits of 1 unit for each additional hospital to which they are assigned. In particular, we modified the base case by adding more constraints with some new decision variables to keep track of the distribution of Specialists of Type B, with the goal of maximize benefits for the five hospitals. The extension case proceeds with the base case where we applied an upper bound on the number of Nurse Practitioners assigned to each hospital.

Mathematical Formulation

Let the following be additional decision variables:

$$\beta_{ij} = \begin{cases} 1 & \text{the } j^{th} \text{ Specialist of Type } B \text{ is assigned to hospital } i \\ 0 & \text{the } j^{th} \text{ Specialist of Type } B \text{ is not assigned to hospital } i \end{cases}$$

$$P_j = \text{number of penalty } j^{th} \text{ Specialist Type B received for working in more than one hospital}$$

$$\forall i = 1, 2, \cdots, 5 \text{ and } \forall j = 1, 2, 3$$

To model our extension, we removed constraint (5) from the original model because Specialist of Type B may be assigned to more than one hospital. Two new constraints were introduced to keep track of the number of Specialists of Type B assigned to hospital i and the penalty accumulated by each Specialist of Type B.

Constraint (12) decides which indicator variable y_{Bij} needs to be turned on for each hospital i:

$$\sum_{j=1}^{3} \beta_{ij} = \sum_{j=1}^{3} j \cdot y_{Bij}, \qquad \forall i = 1, 2, \cdots, 5 \tag{12}$$

Constraint (13) computes the penalty accumulated by the jth Specialist of Type B considering the penalty is a decrease of 1 unit of benefit for each additional hospital they are assigned to:

$$P_i = \sum_{i=1}^{5} \beta_{ij} - 1, \qquad \forall j = 1, 2, 3 \tag{13}$$

We also modified our objective function by attaching the penalty terms:

$$Z_{new} = Z_{original} - \sum_{j=1}^{3} P_i$$

Result and Discussion

We modified the base case problem where we had the upper bounds on the distribution of Nurse Practitioners and solved it using `MATLAB`. The maximized benefit is $Z = 295$ and the health care workers' allocations are enlisted in Table 5. The assignments of Specialists of Type A and D do not differ from what we got in the base case. However, changing the constraints on Specialists of Type B also change the assignments of Specialists of Type C because of the constraint that at least one of them must be assigned to each hospital.

Worker / Hospital	Specialist Type				Nurse Practitioner
	A	B	C	D	
1	2	3	4	1	50
2	2	3	0	1	50
3	2	3	0	1	50
4	1	3	0	0	50
5	1	3	0	0	50

Table 5: The allocations of health care workers to the hospitals with penalty of 1

Since we allowed the hospitals to take as many Specialists of Type B as possible, the problem results in assigning all 3 available Specialists of Type B to each hospital. This is justified because assigning 3 Specialists of Type B to a hospital has a net benefit of 2, while in the base case, assigning 1 Specialist of Type B to a hospital, though has zero penalty, only gives a net benefit of 1 to that hospital.

This results might not be reasonable in real life since assigning a worker to 5 different hospital might brought some logistic issues. Since the problem was posed by having small penalty, we can examine this problem by giving a larger value for the penalty. Note that when the penalty is too large, the solutions will favor assigning 1 specialist to a hospital. Thus, we chose a penalty of 2

where it gives us a good in the middle ground between maximizing the number of hospitals that have Specialist of Type B, but not overworking all the specialists.

By increasing the penalty to 2 instead of 1, the optimal solution is to have 1 Specialist of Type B work in an additional hospital. And, Specialists of Type C are again all assigned to one hospital. The decision variables are included in Table 6. Correspondingly, the maximized benefits is $Z = 284$, one higher than in the base case. The results of the two situations are also presented in Figure 2.

Hospital \ Worker	Specialist Type				Nurse Practitioner
	A	B	C	D	
1	2	1	0	1	50
2	2	1	0	1	50
3	2	1	0	1	50
4	1	1	0	0	50
5	1	0	4	0	50

Table 6: The allocations of health care workers to the hospitals with penalty of 2

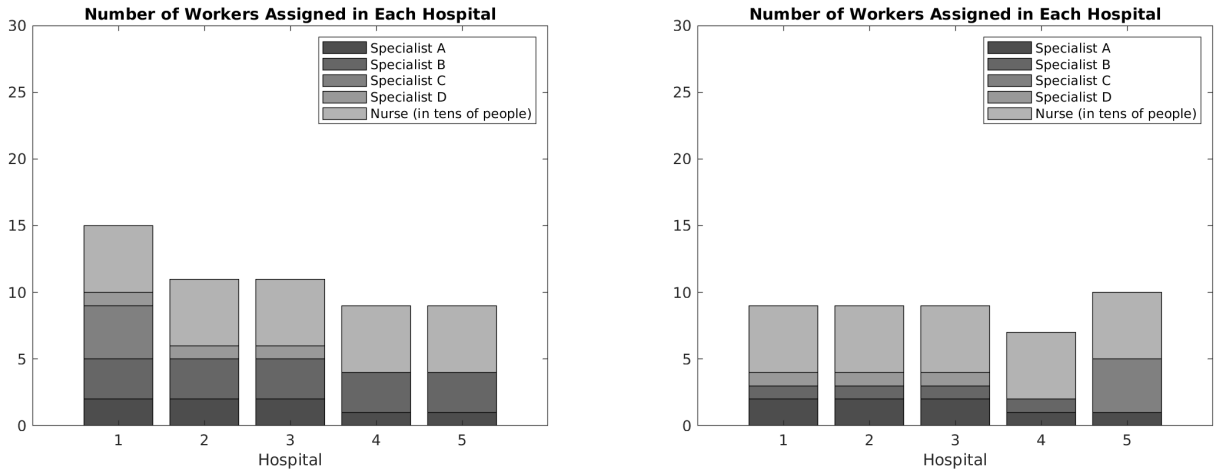

Figure 2: Allocation of workers for each hospital, (nurses are counted in unit of tens of people). Left : with the original problem penalty. Right : result from increasing the value of penalty

Conclusion

In this report, we formulated an assignment problem as an Integer Linear Programming (ILP) problem where we tried to maximize the benefits of allocating 4 different types of Specialists and Nurse Practitioners to 5 different hospitals. We worked on a base case and an extension case of the problem.

Our base case was constructed based on some requirements for the number of workers that each hospital had to meet. The ILP problem was solved using `MATLAB` and a maximized benefits of 283 was obtained. There were two interesting observations that we found from the results: the solutions

were not unique and the benefits for each hospital were unequally distributed. To alleviate the disparity of benefits between each hospital, we added an upper bound constraint on the number of the Nurse Practitioners to obtain a better distribution of them that still have an optimal benefit of 283.

Our second case was the extension of the first case where we then assumed we could assign Specialists of Type B to more than one hospital, but penalize the benefit by 1 for each additional hospital they were assigned to. We obtained a benefit of 295 from solving the ILP problem again using `MATLAB`. This result might not be viable in real life, which motivated us to increases the penalty term to be 2. With this model, a benefits of 284 was earned, slightly higher than what we got from the base case, but one additional hospital got assigned a Specialist of Type B. In its way, increasing the penalty to 2 acted as a trade-off between forcing all Specialists of Type B to work in every hospital and having them at as many hospitals as possible.

As we discussed previously, we noticed that the benefits of each hospital were unequally distributed. We proposed a solution by assigning an upper bound of 50 to the number of Nurse Practitioners that can be assigned to each hospital. Although this method works for this particular case, we believe that there is a better approach that could work on a more general case. One such solution is a bi-criteria Integer Linear Programming model. We can introduce new decision variables that penalize the difference between the benefits among the hospitals. Our objective function thus would be to maximize the benefits of assigning health care workers to all hospitals while trying to minimize the penalty accrued from the difference in benefits amongst them.

Appendix

Base Case Code

```
function[]=FinalProject()
%This is the function to solve MAT 3810  Final Project
%=================List of Variable====================
% nConstraintLEQ is how many less or equal constraint we have (# of Row
% nConstraintEQ is how many  equality constraint we have (# of Row
% aafALEQ is the less or equal constraint matrix
% afbRHSLEQ is the array that saves the right hand side of less or equal to
% aafAEQ is th equality constraints
% afBRHSEQ is the right hand side of equality constraint
% nLEQCounter counter for less pr equal constraints
% nEQCounter counter for equality constraints
% afLB is the lowerbound array of DV's
% afUB is the upperbound array of DV's
% optDV  optimal weights of cargo 1,2,3,4 assign to which part of the plane
% optZ optimal Z profit of the plane
% C is coeffiecient of Z
% nDV is the number of decission variable
% afWorkerAvailable = array that stores the number of worker
%                                   available for each specialist
% afSpecialistBenefit = matrix that store the number of benefit for
```

```
%                                   n(row) number of Specialist of type j (Column)
%-----------------------------------------------------------------
% Initializing variables
%-----------------------------------------------------------------

% Number of hospital needed to serve
    nHospital =5 ;
% array stores number of worker available
    afWorkerAvailable = AvailableWorker();
%matrix store the benefit of worker
    aafSpecialistBenefit = SpecialistBenefit();
%finding number of decission variable needed
    nDV = nHospital*(sum(afWorkerAvailable(1:3))+1+1);
% Number of  inequality constraint
    nConstraintLEQ= 20;
%Number of equality constraint
    nConstraintEQ = 10;
% number of each medical worker available
        nSpecialistA = afWorkerAvailable(1);
        nSpecialistB = afWorkerAvailable(2);
        nSpecialistC = afWorkerAvailable(3);
        nSpecialistD = afWorkerAvailable(4);
        nNurse = afWorkerAvailable(5);
        nTypeWorker = size(afWorkerAvailable,2);
%Initialize aafALEQ, afbRHSLEQ,nLEQCounter
    aafALEQ = zeros(nConstraintLEQ,nDV);
    afbRHSLEQ = zeros(nConstraintLEQ,1);
    nLEQCounter = 0 ;
 %Initialize aafALEQ, afbRHSLEQ,nLEQCounter
    aafAEQ = zeros(nConstraintEQ,nDV);
    afbRHSEQ = zeros(nConstraintEQ,1);
    nEQCounter = 0 ;
% Initialize  the upperBound
    afUB=ones(1,nDV);
%Initialize the lower bound
    afLB =zeros(1,nDV);
%Initialize objective function
     C=zeros(1,nDV);
%initialize aafAns
    aafAns = zeros(nHospital*(nTypeWorker),nDV);
%initialize afAns
    afAnswer = zeros(nHospital*(nTypeWorker),1);

%-------------------------------------------------------------------
%Building LookUp and reverse lookup table
%-------------------------------------------------------------------
```

```
  [SpecialistALookUp,SpecialistAReverseLookUp,nSpecialistASolDV,...
    SpecialistBLookUp,SpecialistBReverseLookUp,nSpecialistBSolDV,...
    SpecialistCLookUp,SpecialistCReverseLookUp,nSpecialistCSolDV,...
    SpecialistDLookUp,SpecialistDReverseLookUp,nSpecialistDSolDV,...
    NurseLookUp,NurseReverseLookUp, nNurseSolDV] =...
        buildLookUp(nHospital,nSpecialistA,...
        nSpecialistB,nSpecialistC,nSpecialistD,nNurse);

%-----------------------------------------------------------------
% Build Constraint Matrices
%-----------------------------------------------------------------
 [aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,nLEQCounter,nEQCounter]=...
    buildConstraint(nHospital,nDV,aafSpecialistBenefit,...
    nSpecialistA,nSpecialistB,...
    nSpecialistC,nSpecialistD,nNurse,...
    SpecialistALookUp,...
    SpecialistAReverseLookUp,nSpecialistASolDV,...
    SpecialistBLookUp,...
    SpecialistBReverseLookUp,nSpecialistBSolDV,...
    SpecialistCLookUp,...
    SpecialistCReverseLookUp,nSpecialistCSolDV,...
    SpecialistDLookUp,...
    SpecialistDReverseLookUp,nSpecialistDSolDV,...
    NurseLookUp,NurseReverseLookUp, nNurseSolDV,...
    aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,nLEQCounter,nEQCounter) ;

%-----------------------------------------------------------------
%                            Upper and Bound
%-----------------------------------------------------------------
% Create Upper Bound
    [afUB] = createUpperBound(afUB,nSpecialistDSolDV,nDV);
%Create Lower Bound
    [afLB] = createLowerBound(afLB,nSpecialistDSolDV,nDV);

%-----------------------------------------------------------------
%Set which one is integer variable
%-----------------------------------------------------------------
 intcon =[1:nDV];

%-----------------------------------------------------------------
%                     Objective Function
%-----------------------------------------------------------------

[C] = buildObjectives(...
        C,nDV,aafSpecialistBenefit,...
        SpecialistALookUp,...
        SpecialistAReverseLookUp,nSpecialistASolDV,...
```

```
        SpecialistBLookUp,...
        SpecialistBReverseLookUp,nSpecialistBSolDV,...
        SpecialistCLookUp,...
        SpecialistCReverseLookUp,nSpecialistCSolDV,...
        SpecialistDLookUp,...
        SpecialistDReverseLookUp,nSpecialistDSolDV,...
        NurseLookUp,NurseReverseLookUp, nNurseSolDV);

%-----------------------------------------------------------------
%Solve our model
%-----------------------------------------------------------------
 [optDVIP,optZIP] = intlinprog(-C,intcon,...
                                    aafALEQ,afbRHSLEQ,...
                                    aafAEQ,afbRHSEQ,...
                                    afLB,afUB);


%-----------------------------------------------------------------
%Building answer matrix
%-----------------------------------------------------------------
 [aafAns] = buildAnswer(nDV,nHospital,...
            SpecialistALookUp,nSpecialistA,...
            SpecialistBLookUp,nSpecialistB,...
             SpecialistCLookUp,nSpecialistC,...
             SpecialistDLookUp,nSpecialistD,...
             NurseLookUp, nNurse,...
             aafAns);
 afAllocation= aafAns*optDVIP ;% nurse values in tens of people
%Print answer
    %Print Allocation
       %Specialist A
            nCount = 0 ;
            for i =1 :nHospital
                 nCount =nCount+1;
                 fprintf('Hospital %d have %d Specialist Type A \n',...
                     i,afAllocation(nCount))
            end
      %Specialist Type B
            for i =1 :nHospital
                 nCount =nCount+1;
                 fprintf('Hospital %d have %d Specialist Type B \n',...
                     i,afAllocation(nCount))
            end
        %Specialist Type C
            for i =1 :nHospital
                 nCount =nCount+1;
                 fprintf('Hospital %d have %d Specialist Type C \n',...
```

```
                    i,afAllocation(nCount))
            end
          %Specialist Type D
                for i =1 :nHospital
                     nCount =nCount+1;
                     fprintf('Hospital %d have %d Specialist Type D \n',...
                         i,afAllocation(nCount))
                end
          %Nurse
                for i =1 :nHospital
                     nCount =nCount+1;
                     fprintf('Hospital %d have %d Nurse \n',...
                         i,10*afAllocation(nCount))
                end
 makeBarPlot(nHospital,nTypeWorker,afAllocation)

end


function[A]=AvailableWorker ()
%This function is to build a matrix that provides information about the
%number of worker available for specialist A,B,C,D, and Nurse Practioner
% 8 Specialist A
% 3 Specialist  B
% 4 Specialist C
% 3 Specialist D
% 250 Nurse Practitioner
A= [8,3,4,3,250];
end


function[A]=SpecialistBenefit()
%This function is to build a matrix that stores
%information about the
%Benefit of worker per specialist assigned to hospital
%row represent number of specialist assigned
%column represent type of specialisr
%       Column 1  = Specialist 1
%       Column 2  = Specialist 2
%       Column 3  = Specialist 3
%       Column 4  = Specialist 4
%       Column 5  = Nurse Practitioner

A=[0,1,0,2;...
      7,2,1,4;...
      11,5,3,9;...
      14,8,5,20;...
```

```
      16,13,8,43;...
      18,18,12,90;...
      19,25,16,191;...
      21,32,21,403];
end


function[A,nLookUpCounter,B]=...
    BuildSpecialistALookup(nHospital,nSpecialistA,nLookUpCounter)
%This is function build the look up table and reverse Look up table

%==========Output  Variable======
% Matrix A for tracking
% whether j number of specialist A is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is equal to which
% j number of speacialist A and hospital i

% nLookUp Counter to track how many DV Specialist A use

%==========input variable========
%nHospital =  number of hospital to be assigned
%nSpecialistA = number of Specialist A available to be assigned

%Initialize Look up table
    A= zeros(nHospital,nSpecialistA);
    B= zeros(nHospital*nSpecialistA,2);
%Assigning value
    for i = 1:nHospital
        for j = 1:nSpecialistA
            nLookUpCounter = nLookUpCounter+1;
            A(i,j) = nLookUpCounter;
            B(nLookUpCounter,:) = [i,j];
        end
    end
end


function[A,nLookUpCounter,B]=...
    BuildSpecialistBLookup(nHospital,nSpecialistB,nLookUpCounter)
%This is function build the look up table and reverse Look up table

%==========Output  Variable===============
% Matrix A for tracking whether j number of specialist B is assign to hospital i
% 0 if its not assigned and 1 if it's assigned
```

```
%Matrix B for tracking binary DV variable is equal to which
% j(column) number of speacialist B and hospital i (row)

% nLookUp Counter to track how many DV Specialist B use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistB = number of Specialist B available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended
%Initialize Look up table
    A= zeros(nHospital,nSpecialistB);
    B= zeros(nHospital*nSpecialistB,2);
%Assigning value
nCount = 0;
    for i = 1:nHospital
        for j = 1:nSpecialistB
            nCount = nCount+1;
            A(i,j) = nLookUpCounter+nCount;
            B(nCount,:) = [i,j];
        end
    end
%Update nLookUp Cpunter
    nLookUpCounter = nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B]=...
    BuildSpecialistCLookup(nHospital,nSpecialistC,nLookUpCounter)
%This is function build the look up table and
%reverse Look up table

%==========Output  Variable======
% Matrix A for tracking whether j number of specialist C is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is equal to which
% j(column) number of speacialist C and hospital i (row)

% nLookUp Counter to track how many DV Specialist C use

%==========input variable========
%nHospital =  number of hospital to be assigned
%nSpecialistC = number of Specialist C available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended

%Initialize Look up table and reverse  lookup table
    A= zeros(nHospital,nSpecialistC);
```

```
    B= zeros(nHospital*nSpecialistC,2);
%Assigning value
nCount = 0;
    for i = 1:nHospital
        for j = 1:nSpecialistC
            nCount = nCount+1;
            A(i,j) = nLookUpCounter+nCount;
            B(nCount,:) = [i,j];
        end
    end
%Update nLookUp Cpunter
    nLookUpCounter = nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B] = ...
    BuildSpecialistDLookup(nHospital,nLookUpCounter)
%This is function build the look up table and reverse Look up table

%==========Output  Variable=======
% Matrix A for tracking whether specialist D is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is related to which Specialist D
%is assigned to hospital i

% nLookUp Counter to track how many DV Specialist C use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistC = number of Specialist C available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended

%Initialize Look up table and reverse  lookup table
A =zeros(1,nHospital);
B=zeros(1,nHospital);

% Assigning value to look up tables
nCount  = 0;
for i = 1:nHospital
    nCount = nCount+1;
    A(i)= nLookUpCounter+nCount;
    B(i)= nCount;
end
nLookUpCounter= nLookUpCounter+nCount;
end
```

```
function[A,nLookUpCounter,B] = ...
    BuildNurseLookup(nHospital,nLookUpCounter)
%This is function build the look up table and reverse Look up table

%==========Output  Variable======
% Matrix A for tracking the number
%(integer of nurse  is assigned  to hospital i

%Matrix B for tracking DV sol is related to nurse from hospital i

% nLookUp Counter to track how many DV Specialist C use

%==========input variable========
%nHospital =  number of hospital to be assigned
%nSpecialistC = number of Specialist C available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended

%Initialize Look up table and reverse  lookup table
A =zeros(1,nHospital);
B=zeros(1,nHospital);

% Assigning value to look up tables
nCount  = 0;
for i = 1:nHospital
    nCount = nCount+1;
    A(i)= nLookUpCounter+nCount;
    B(i)= nCount;
end
nLookUpCounter= nLookUpCounter+nCount;
end

function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
    Constraint1(nHospital,nDV,nSpecialistA,SpecialistALookUp,...
    aafALEQ,afbRHSLEQ,nLEQCounter)

for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        NewConstraint=zeros(1,nDV);
        for j = 1:nSpecialistA
            NewConstraint(SpecialistALookUp(i,j)) = -j ;
        end
        NewConstraint;
        aafALEQ(nLEQCounter,:) =NewConstraint;
        afbRHSLEQ(nLEQCounter)=-1;
    end
end
```

```
function [aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint2(nHospital,nSpecialistB,SpecialistBLookUp,...
       aafALEQ,afbRHSLEQ,nLEQCounter)
    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        for j = 1:nSpecialistB
            aafALEQ (nLEQCounter,SpecialistBLookUp(i,j)) = 1;
            afbRHSLEQ (nLEQCounter) = 1;
        end
    end
end

function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint3(nHospital,nSpecialistC,SpecialistCLookUp,...
       aafALEQ,afbRHSLEQ,nLEQCounter)
    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        for j = 1:nSpecialistC
            aafALEQ (nLEQCounter,SpecialistCLookUp(i,j)) = 1;
            afbRHSLEQ (nLEQCounter) = 1;
        end
    end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter] =...
       Constraint4(nHospital,nDV,nSpecialistC,SpecialistCLookUp,...
       nSpecialistB,SpecialistBLookUp,aafALEQ,afbRHSLEQ,nLEQCounter)

       for i = 1:nHospital
           nLEQCounter = nLEQCounter+1;
           NewConstraint = zeros(1,nDV);
           for j =1:nSpecialistB
               NewConstraint(SpecialistBLookUp(i,j)) = -j ;
           end
           for j = 1:nSpecialistC
               NewConstraint(SpecialistCLookUp(i,j)) = -j ;
           end
           %NewConstraint
           aafALEQ(nLEQCounter,:) = NewConstraint;
           afbRHSLEQ(nLEQCounter) = -1;
       end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
```

```
        Constraint5(nHospital,nDV,nSpecialistD,SpecialistDLookUp,...
        aafALEQ,afbRHSLEQ,nLEQCounter)

    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        NewConstraint = zeros(1,nDV);
        NewConstraint(SpecialistDLookUp(i)) = 1;
        aafALEQ(nLEQCounter,:) = NewConstraint;
        afbRHSLEQ(nLEQCounter) = 1;
    end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
        Constraint6(nHospital,nDV,nNurse,NurseLookUp,...
        aafALEQ,afbRHSLEQ,nLEQCounter)

    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        NewConstraint = zeros(1,nDV);
        NewConstraint(NurseLookUp(i)) = -1;
        aafALEQ(nLEQCounter,:) = NewConstraint;
        afbRHSLEQ(nLEQCounter) = -20;
    end
end


function[aafAEQ,afbRHSEQ,nEQCounter]=...
    Constraint7(nHospital,nDV,nSpecialistA,SpecialistALookUp,...
    aafAEQ,afbRHSEQ,nEQCounter)
         for i = 1:nHospital
             nEQCounter = nEQCounter+1;
             NewConstraint = zeros(1,nDV);
             for j = 1:nSpecialistA
                 NewConstraint(SpecialistALookUp(i,j))=1;
             end
             %NewConstraint;
             aafAEQ(nEQCounter,:)= NewConstraint;
             afbRHSEQ(nEQCounter) =1;
         end

end


function[aafAEQ,afbRHSEQ,nEQCounter]=...
        Constraint8(nHospital,nDV,nSpecialistA,SpecialistALookUp,...
        aafAEQ,afbRHSEQ,nEQCounter)
```

```
                nEQCounter = nEQCounter+1;
                NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                     for j = 1:nSpecialistA
                        NewConstraint(SpecialistALookUp(i,j))=j;
                     end
                end
            %NewConstraint;
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =8;
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint9(nHospital,nDV,...
       nSpecialistC,SpecialistCLookUp,...
       nSpecialistB,SpecialistBLookUp,...
       aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                     for j = 1:nSpecialistB
                        NewConstraint(SpecialistBLookUp(i,j))=j;
                     end
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =3;
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint10(nHospital,nDV,nSpecialistC,SpecialistCLookUp,...
       nSpecialistB,SpecialistBLookUp,...
       aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                     for j = 1:nSpecialistC
                        NewConstraint(SpecialistCLookUp(i,j))=j;
                     end
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =4;
```

```
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint11(nHospital,nDV,...
       nSpecialistD,SpecialistDLookUp,...
       aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                        NewConstraint(SpecialistDLookUp(i))=1;
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =3;
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint12(nHospital,nDV,...
       nNurse,NurseLookUp,...
       aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                        NewConstraint(NurseLookUp(i))=1;
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =250;
end


function[SpecialistALookUp,...
    SpecialistAReverseLookUp,nSpecialistASolDV,...
    SpecialistBLookUp,...
    SpecialistBReverseLookUp,nSpecialistBSolDV,...
    SpecialistCLookUp,...
    SpecialistCReverseLookUp,nSpecialistCSolDV,...
    SpecialistDLookUp,...
    SpecialistDReverseLookUp,nSpecialistDSolDV,...
    NurseLookUp,NurseReverseLookUp,nNurseSolDV] =...
        buildLookUp(nHospital,nSpecialistA,nSpecialistB,...
        nSpecialistC,nSpecialistD,nNurse )
```

```
%Initialize DV counter
    nLookUpCounter = 0 ;
%Create Look up and Reverse Look for Specialist A
    [SpecialistALookUp,nLookUpCounter,...
        SpecialistAReverseLookUp] = ...
        BuildSpecialistALookup(nHospital,nSpecialistA,nLookUpCounter);
%Track where Specialist A DV ends
    nSpecialistASolDV = nLookUpCounter;
%Create Lookup and reverse Lookup for Specialist B
    [SpecialistBLookUp,nLookUpCounter,...
        SpecialistBReverseLookUp] = ...
    BuildSpecialistBLookup(nHospital,nSpecialistB,nLookUpCounter);
%Track where Specialist B DV ends
    nSpecialistBSolDV = nLookUpCounter;
%Build LookUp and Reverse Look up table for Specialist C
    [SpecialistCLookUp,nLookUpCounter,...
        SpecialistCReverseLookUp] = ...
        BuildSpecialistCLookup(nHospital,nSpecialistC,nLookUpCounter);
%Track Where Specialist C DV ends
    nSpecialistCSolDV = nLookUpCounter;
%Build LookUp and Reverse Look up table for Specialist D
    [SpecialistDLookUp,nLookUpCounter,SpecialistDReverseLookUp] = ...
        BuildSpecialistDLookup(nHospital,nLookUpCounter);
%Track where Specialist D DV  ends
    nSpecialistDSolDV = nLookUpCounter;
%Build LookUp and Reverse Look up table for Nurse Practitioner
    [NurseLookUp,nLookUpCounter,NurseReverseLookUp] = ...
        BuildNurseLookup(nHospital,nLookUpCounter);
%Track where Specialist D DV  ends
    nNurseSolDV = nLookUpCounter;
end


function[aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,nLEQCounter,nEQCounter]=...
    buildConstraint(nHospital,nDV,aafSpecialistBenefit,...
        nSpecialistA,...
        nSpecialistB,nSpecialistC,nSpecialistD,nNurse,...
        SpecialistALookUp,...
        SpecialistAReverseLookUp,nSpecialistASolDV,...
        SpecialistBLookUp,...
        SpecialistBReverseLookUp,nSpecialistBSolDV,...
        SpecialistCLookUp,...
        SpecialistCReverseLookUp,nSpecialistCSolDV,...
        SpecialistDLookUp,...
        SpecialistDReverseLookUp,nSpecialistDSolDV,...
        NurseLookUp,NurseReverseLookUp, nNurseSolDV,...
        aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,nLEQCounter,nEQCounter)
```

```
%-------------------------------------------------------------
%            Less or equal Constraints
%-------------------------------------------------------------

% at least one specialist A in each hospital
    %dont need this because constraint 7 guarantees constraint 1
%     [aafALEQ,afbRHSLEQ,nLEQCounter]=...
%        Constraint1(nHospital,nDV,nSpecialistA,SpecialistALookUp,aafALEQ,afbRHSLEQ,nLEQCou

% can only pick berween 1,2,or 3 for specialist B in each hospital
    [aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint2(nHospital,nSpecialistB,SpecialistBLookUp,...
       aafALEQ,afbRHSLEQ,nLEQCounter);
%can only pick between 1,2,3,4 for specialist B in each hospital
    [aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint3(nHospital,nSpecialistC,SpecialistCLookUp,...
       aafALEQ,afbRHSLEQ,nLEQCounter);
%atleast one specialist B or C in each hospital
   [aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint4(nHospital,nDV,nSpecialistC,SpecialistCLookUp,...
       nSpecialistB,SpecialistBLookUp,aafALEQ,afbRHSLEQ,nLEQCounter);



%-------------------------------------------------------------
%                  Equality Constraints
%-------------------------------------------------------------


%can either pick 1,2 ,3 ,..8 for Specalist A in each hospital
    [aafAEQ,afbRHSEQ,nEQCounter]=...
       Constraint7(nHospital,nDV,nSpecialistA,SpecialistALookUp...
       ,aafAEQ,afbRHSEQ,nEQCounter);
  % can only assigned 8 Specialist A in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter]=...
       Constraint8(nHospital,nDV,nSpecialistA,SpecialistALookUp,...
       aafAEQ,afbRHSEQ,nEQCounter);
   %can  only assigned 3 Specialist B in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint9(nHospital,nDV,nSpecialistC,SpecialistCLookUp,...
       nSpecialistB,SpecialistBLookUp,aafAEQ,afbRHSEQ,nEQCounter);
    % can only assigned 4 specialist C in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint10(nHospital,nDV,nSpecialistC,SpecialistCLookUp,...
       nSpecialistB,SpecialistBLookUp,aafAEQ,afbRHSEQ,nEQCounter);
   %Can only assigned 3 Specialist D in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter] =...
```

```
        Constraint11(nHospital,nDV,nSpecialistD,SpecialistDLookUp,...
        aafAEQ,afbRHSEQ,nEQCounter);
    % Can only assigned 250 Nurses in 5 hospital
    [aafAEQ,afbRHSEQ,nEQCounter] =...
        Constraint12(nHospital,nDV,nNurse,NurseLookUp,...
        aafAEQ,afbRHSEQ,nEQCounter);

 end



function[afUB] = createUpperBound(afUB,nSpecialistDSolDV,nDV)
         %upper bound for Nurse
            for i=nSpecialistDSolDV+1:nDV
               afUB(i) =50;
            end
end


function[afLB] = createLowerBound(afLB,nSpecialistDSolDV,nDV)
    %Lower bound for nurses
         for i=nSpecialistDSolDV+1:nDV
           afLB(i) =20;
        end
end


function[C] = buildObjectives(...
            C,nDV,aafSpecialistBenefit,...
            SpecialistALookUp,...
            SpecialistAReverseLookUp,nSpecialistASolDV,...
            SpecialistBLookUp,...
            SpecialistBReverseLookUp,nSpecialistBSolDV,...
            SpecialistCLookUp,...
            SpecialistCReverseLookUp,nSpecialistCSolDV,...
            SpecialistDLookUp,...
            SpecialistDReverseLookUp,nSpecialistDSolDV,...
            NurseLookUp,NurseReverseLookUp, nNurseSolDV)

        NewConstraint= zeros(1,nDV);

        for i=1:nSpecialistASolDV
            NewConstraint(i) =...
                aafSpecialistBenefit(...
                SpecialistAReverseLookUp(i,2),1);
        end
         for i=nSpecialistASolDV+1:nSpecialistBSolDV
```

```
            NewConstraint(i) =...
                aafSpecialistBenefit(...
                SpecialistBReverseLookUp(i-nSpecialistASolDV,2),2);
        end
        for i=nSpecialistBSolDV+1:nSpecialistCSolDV
            NewConstraint(i) =...
                aafSpecialistBenefit(...
                SpecialistCReverseLookUp(i-nSpecialistBSolDV,2),3);
        end
        for i=nSpecialistCSolDV+1:nSpecialistDSolDV
            NewConstraint(i) =2;
        end
        for i=nSpecialistDSolDV+1:nDV
            NewConstraint(i) =1;
        end
        C(1,:)=NewConstraint;
    end

function[aafAns] = buildAnswer(nDV,nHospital,...
                                    SpecialistALookUp,nSpecialistA,...
                                    SpecialistBLookUp,nSpecialistB,...
                                     SpecialistCLookUp,nSpecialistC,...
                                     SpecialistDLookUp,nSpecialistD,...
                                     NurseLookUp, nNurse,...
                                     aafAns)
        nAnsCount = 0 ;

    %Specialist A
        for  i = 1:nHospital
            NewConstraint = zeros(1,nDV);
            nAnsCount = nAnsCount +1 ;
            for j = 1:nSpecialistA
                NewConstraint(SpecialistALookUp(i,j)) = j ;
            end
            aafAns(nAnsCount,:) = NewConstraint;
        end

    %Specialist B
         for  i = 1:nHospital
            NewConstraint = zeros(1,nDV);
            nAnsCount = nAnsCount +1 ;
            for j = 1:nSpecialistB
                NewConstraint(SpecialistBLookUp(i,j)) = j ;
            end
            aafAns(nAnsCount,:) = NewConstraint;
         end
```

```
    %Specialist C
         for  i = 1:nHospital
            NewConstraint = zeros(1,nDV);
            nAnsCount = nAnsCount +1 ;
            for j = 1:nSpecialistC
                NewConstraint(SpecialistCLookUp(i,j)) = j ;
            end
            aafAns(nAnsCount,:) = NewConstraint;
         end

    %Specialist D
        for i = 1 :nHospital
            nAnsCount=nAnsCount+1;
            aafAns(nAnsCount,SpecialistDLookUp(i)) = 1;
        end

     %NursePractitioner
        for i = 1 :nHospital
            nAnsCount=nAnsCount+1;
            aafAns(nAnsCount,NurseLookUp(i)) = 1/10;%nurse in tens of people
        end
end


function[]=makeBarPlot(nHospital,nTypeWorker,afAnswer)
% Preparing data for bar plot
    aafBarMatrix = zeros(nHospital,nTypeWorker);
    nCounter = 0 ;
     for i = 1:nHospital
         for j = 1:nTypeWorker
             nCounter = nCounter+1;
             aafBarMatrix(j,i) = afAnswer(nCounter);
         end
     end
     aafBarMatrix;

%-----------------------------------------------------------------
%Creating Bar Plot
%-----------------------------------------------------------------

    figure()
    b = bar(aafBarMatrix,'stacked');

    b(1).FaceColor = [0.3,0.3,0.3];
    b(2).FaceColor = [0.4,0.4,0.4];
    b(3).FaceColor = [0.5,0.5,0.5];
    b(4).FaceColor = [0.6,0.6,0.6];
```

```
    b(5).FaceColor = [0.7,0.7,0.7];
    xlabel ('Hospital')
    cTitle = ['Number of Workers Assigned in Each Hospital'];
    title(cTitle);
    xlim([0,6]);
    ylim([0,23]);

    legend('Specialist A','Specialist B', ...
        'Specialist C', 'Specialist D','Nurse (in tens of people')
 end
```

```
LP:                Optimal objective value is -287.000000.

Cut Generation:    Applied 1 clique cut, and 1 Gomory cut.
                   Lower bound is -283.000000.
                   Relative gap is 0.00%.


Optimal solution found.

Intlinprog stopped at the root node because the objective value is within a gap
tolerance of the optimal value, options.AbsoluteGapTolerance = 0 (the default
value). The intcon variables are integer within tolerance,
options.IntegerTolerance = 1e-05 (the default value).

Hospital 1 have 2 Specialist Type A
Hospital 2 have 2 Specialist Type A
Hospital 3 have 2 Specialist Type A
Hospital 4 have 1 Specialist Type A
Hospital 5 have 1 Specialist Type A
Hospital 1 have 1 Specialist Type B
Hospital 2 have 0 Specialist Type B
Hospital 3 have 1.000000e+00 Specialist Type B
Hospital 4 have 1.000000e+00 Specialist Type B
Hospital 5 have 0 Specialist Type B
Hospital 1 have 0 Specialist Type C
Hospital 2 have 1 Specialist Type C
Hospital 3 have 0 Specialist Type C
Hospital 4 have 0 Specialist Type C
Hospital 5 have 3.000000e+00 Specialist Type C
Hospital 1 have 1 Specialist Type D
Hospital 2 have 1 Specialist Type D
Hospital 3 have 1 Specialist Type D
Hospital 4 have 0 Specialist Type D
Hospital 5 have 0 Specialist Type D
Hospital 1 have 50 Nurse
```

```
Hospital 2 have 50 Nurse
Hospital 3 have 50 Nurse
Hospital 4 have 50 Nurse
Hospital 5 have 50 Nurse
```

Extension Case Code

```
function[]=FinalProject()
%This is the function to solve MAT 3810  Final Project
%=================List of Variable====================
% nConstraintLEQ is how many less or equal constraint we have (# of Row
% nConstraintEQ is how many  equality constraint we have (# of Row
% aafALEQ is the less or equal constraint matrix
% afbRHSLEQ is the array that saves the right hand side of less or eaual to
% aafAEQ is th equality constraints
% afBRHSEQ is the right hand side of equality constraint
% nLEQCounter counter for less pr equal constraints
% nEQCounter counter for equality constraints
% afLB is the lowerbound array of DV's
% afUB is the upperbound array of DV's
% optDV  optimal weights of cargo 1,2,3,4 assign to which part of the plane
% optZ optimal Z profit of the plane
% C is coeffiecient of Z
%nDV is the number of decission variable
%afWorkerAvailable = array that stores the
%       number of worker available for each specialist
%afSpecialistBenefit = matrix that store the number of
%       benefit for n(row) number of Specialist of type j (Column)


%-----------------------------------------------------------------
% Initializing variables
%-----------------------------------------------------------------

% Number of hospital needed to serve
    nHospital =5 ;
%Penalty for each worker B that work more than 1 hospital
    nPenalty =2;
% array stores number of worker available
    afWorkerAvailable = AvailableWorker();
%matrix store the benefit of worker
    aafSpecialistBenefit = SpecialistBenefit();
%finding number of decission variable needed
    nDV = nHospital*(sum(afWorkerAvailable(1:3))+...
        1+1+afWorkerAvailable(2))+afWorkerAvailable(2);
%penalty
```

```
% Number of  inequality constraint
    nConstraintLEQ= 20;
%Number of equality constraint
    nConstraintEQ = 10;
% number of each medical worker available
        nSpecialistA = afWorkerAvailable(1);
        nSpecialistB = afWorkerAvailable(2);
        nSpecialistC = afWorkerAvailable(3);
        nSpecialistD = afWorkerAvailable(4);
        nNurse = afWorkerAvailable(5);
        nTypeWorker = size(afWorkerAvailable,2);
%Initialize aafALEQ, afbRHSLEQ,nLEQCounter
    aafALEQ = zeros(nConstraintLEQ,nDV);
    afbRHSLEQ = zeros(nConstraintLEQ,1);
    nLEQCounter = 0 ;
 %Initialize aafALEQ, afbRHSLEQ,nLEQCounter
    aafAEQ = zeros(nConstraintEQ,nDV);
    afbRHSEQ = zeros(nConstraintEQ,1);
    nEQCounter = 0 ;
% Initialize  the upperBound
    afUB=ones(1,nDV);
%Initialize the lower bound
    afLB =zeros(1,nDV);
%Initialize objective function
     C=zeros(1,nDV);
%initialize aafAns
    aafAns = zeros(nHospital*(nTypeWorker),nDV);
%initialize afAns
    afAnswer = zeros(nHospital*(nTypeWorker),1);
%Initialize Penalty
      afPenalty =zeros(1,3);

%-----------------------------------------------------------------
%Building LookUp and reverse lookup table
%-----------------------------------------------------------------

  [SpecialistALookUp,...
  SpecialistAReverseLookUp,nSpecialistASolDV,...
    SpecialistBLookUp,...
    SpecialistBReverseLookUp,nSpecialistBSolDV,...
    SpecialistCLookUp,...
    SpecialistCReverseLookUp,nSpecialistCSolDV,...
    SpecialistDLookUp,...
    SpecialistDReverseLookUp,nSpecialistDSolDV,...
    NurseLookUp,NurseReverseLookUp, nNurseSolDV,...
    WorkerBLookup,nWorkerSolDV,WorkerBReverseLookUp,...
    PenaltyLookUp,nLookUpCounter,PenaltyReverseLookUp] =...
```

```
    buildLookUp(nHospital,nSpecialistA,nSpecialistB,...
    nSpecialistC,nSpecialistD,nNurse);

%-----------------------------------------------------------------
% Build Constraint Matrices
%-----------------------------------------------------------------
 [aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,nLEQCounter,nEQCounter]=...
    buildConstraint(nHospital,nDV,aafSpecialistBenefit,...
        nSpecialistA,nSpecialistB,...
        nSpecialistC,nSpecialistD,nNurse,...
        SpecialistALookUp,...
        SpecialistAReverseLookUp,nSpecialistASolDV,...
        SpecialistBLookUp,...
        SpecialistBReverseLookUp,nSpecialistBSolDV,...
        SpecialistCLookUp,...
        SpecialistCReverseLookUp,nSpecialistCSolDV,...
        SpecialistDLookUp,...
        SpecialistDReverseLookUp,nSpecialistDSolDV,...
        NurseLookUp,NurseReverseLookUp, nNurseSolDV,...
        WorkerBLookup,nWorkerSolDV,...
        WorkerBReverseLookUp,...
        aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,...
        nLEQCounter,nEQCounter) ;

%-----------------------------------------------------------------
%                            Upper and Bound
%-----------------------------------------------------------------
% Create Upper Bound
    [afUB] = ...
        createUpperBound(afUB,nSpecialistDSolDV,...
        nNurseSolDV,nWorkerSolDV,nDV);
%Create Lower Bound
    [afLB] = createLowerBound(afLB,nSpecialistDSolDV,...
        nNurseSolDV,nDV);


%-----------------------------------------------------------------
%Set which one is integer variable
%-----------------------------------------------------------------
 intcon =[1:nDV];

%-----------------------------------------------------------------
%                     Objective Function
%-----------------------------------------------------------------
[C] = buildObjectives(...
    C,nDV,nPenalty,aafSpecialistBenefit,...
    SpecialistALookUp,...
```

```
    SpecialistAReverseLookUp,nSpecialistASolDV,...
    SpecialistBLookUp,...
    SpecialistBReverseLookUp,nSpecialistBSolDV,...
    SpecialistCLookUp,...
    SpecialistCReverseLookUp,nSpecialistCSolDV,...
    SpecialistDLookUp,...
    SpecialistDReverseLookUp,nSpecialistDSolDV,...
    WorkerBLookup,nWorkerSolDV,WorkerBReverseLookUp,...
    NurseLookUp,NurseReverseLookUp, nNurseSolDV);

%-----------------------------------------------------------------
 %Solve our model
 %-----------------------------------------------------------------
 [optDVIP,optZIP] = intlinprog(-C,intcon,...
                                     aafALEQ,afbRHSLEQ,...
                                     aafAEQ,afbRHSEQ,...
                                     afLB,afUB);

%-----------------------------------------------------------------
%Building answer matrix
%-----------------------------------------------------------------
 [aafAns] = buildAnswer(nDV,nHospital,...
                    SpecialistALookUp,nSpecialistA,...
                    SpecialistBLookUp,nSpecialistB,...
                     SpecialistCLookUp,nSpecialistC,...
                     SpecialistDLookUp,nSpecialistD,...
                     NurseLookUp, nNurse,...
                     aafAns);
 afAllocation= aafAns*optDVIP; % nurse values in tens of people

 %Print answer
    %Print Allocation
       %Specialist A
            nCount = 0 ;
            for i =1 :nHospital
                 nCount =nCount+1;
                 fprintf('Hospital %d have %d Specialist Type A \n',...
                     i,afAllocation(nCount))
            end
      %Specialist Type B
            for i =1 :nHospital
                 nCount =nCount+1;
                 fprintf('Hospital %d have %d Specialist Type B \n',...
                     i,afAllocation(nCount))
            end
        %Specialist Type C
            for i =1 :nHospital
```

```
                 nCount =nCount+1;
                 fprintf('Hospital %d have %d Specialist Type C \n',...
                     i,afAllocation(nCount))
            end
          %Specialist Type D
                for i =1 :nHospital
                     nCount =nCount+1;
                     fprintf('Hospital %d have %d Specialist Type D \n',...
                         i,afAllocation(nCount))
                end
          %Nurse
                for i =1 :nHospital
                     nCount =nCount+1;
                     fprintf('Hospital %d have %d Nurse \n',...
                         i,10*afAllocation(nCount))
                end

    %print where each individual worker of type B
        for i = nNurseSolDV+1:nWorkerSolDV
            if optDVIP(i)==1
                fprintf( 'Specialist type B number %d work in hospital %d \n',...
                WorkerBReverseLookUp(i-nNurseSolDV,2),...
                WorkerBReverseLookUp(i-nNurseSolDV,1))

            end
        end
        %print penalty received from each worker specialist B type
        for i =nWorkerSolDV+1:nDV
            fprintf('penalty from worker % d is %d \n', ...
                PenaltyReverseLookUp(i-nWorkerSolDV),optDVIP(i))
        end

 makeBarPlot(nHospital,nTypeWorker,afAllocation)
% size(aafAns)
% size(optDVIP)
end

function[A]=AvailableWorker ()
%This function is to build a matrix that provides information about the
%number of worker available for specialist A,B,C,D, and Nurse Practioner
% 8 Specialist A
% 3 Specialist  B
% 4 Specialist C
% 3 Specialist D
% 250 Nurse Practitioner
A= [8,3,4,3,250];
end
```

```
function[A]=SpecialistBenefit()
%This function is to build a matrix that stores information about the
%Benefit of worker per specialist assigned to hospital
%row represent number of specialist assigned
%column represent type of specialisr
%       Column 1  = Specialist 1
%       Column 2  = Specialist 2
%       Column 3  = Specialist 3
%       Column 4  = Specialist 4
%       Column 5  = Nurse Practitioner

A=[0,1,0,2;...
      7,2,1,4;...
      11,5,3,9;...
      14,8,5,20;...
      16,13,8,43;...
      18,18,12,90;...
      19,25,16,191;...
      21,32,21,403];
end


function[A,nLookUpCounter,B]=...
    BuildSpecialistALookup(nHospital,nSpecialistA,nLookUpCounter)
%This is function build the look up table and reverse Look up table
%==========Output  Variable===============

% Matrix A for tracking
% whether j number of specialist A is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is equal to which
% j number of speacialist A and hospital i

% nLookUp Counter to track how many DV Specialist A use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistA = number of Specialist A available to be assigned

%Initialize Look up table
    A= zeros(nHospital,nSpecialistA);
    B= zeros(nHospital*nSpecialistA,2);
%Assigning value
    for i = 1:nHospital
```

```
        for j = 1:nSpecialistA
            nLookUpCounter = nLookUpCounter+1;
            A(i,j) = nLookUpCounter;
            B(nLookUpCounter,:) = [i,j];
        end
    end
end


function[A,nLookUpCounter,B]=...
    BuildSpecialistBLookup(nHospital,nSpecialistB,nLookUpCounter)
%This is function build the look up table and reverse Look up table

%==========Output  Variable===============
% Matrix A for tracking whether j number of specialist B is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is equal to which
% j(column) number of speacialist B and hospital i (row)

% nLookUp Counter to track how many DV Specialist B use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistB = number of Specialist B available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended
%Initialize Look up table
    A= zeros(nHospital,nSpecialistB);
    B= zeros(nHospital*nSpecialistB,2);
%Assigning value
nCount = 0;
    for i = 1:nHospital
        for j = 1:nSpecialistB
            nCount = nCount+1;
            A(i,j) = nLookUpCounter+nCount;
            B(nCount,:) = [i,j];
        end
    end
%Update nLookUp Cpunter
    nLookUpCounter = nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B]=...
    BuildSpecialistCLookup(nHospital,nSpecialistC,nLookUpCounter)
%This is function build the look up table and reverse Look up table
```

```
%==========Output  Variable===============
% Matrix A for tracking whether j number of specialist C is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is equal to which
% j(column) number of speacialist C and hospital i (row)

% nLookUp Counter to track how many DV Specialist C use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistC = number of Specialist C available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended

%Initialize Look up table and reverse  lookup table
    A= zeros(nHospital,nSpecialistC);
    B= zeros(nHospital*nSpecialistC,2);
%Assigning value
nCount = 0;
    for i = 1:nHospital
        for j = 1:nSpecialistC
            nCount = nCount+1;
            A(i,j) = nLookUpCounter+nCount;
            B(nCount,:) = [i,j];
        end
    end
%Update nLookUp Cpunter
    nLookUpCounter = nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B] = ...
    BuildSpecialistDLookup(nHospital,nLookUpCounter)
%This is function build the look up table and reverse Look up table

%==========Output  Variable===============
% Matrix A for tracking whether specialist D is assign to hospital i
% 0 if its not assigned and 1 if it's assigned

%Matrix B for tracking binary DV variable is related to which Specialist D
%is assigned to hospital i

% nLookUp Counter to track how many DV Specialist C use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistC = number of Specialist C available to be assigned
```

```
%nLookUpCounter = Keep track of where the previous counter ended

%Initialize Look up table and reverse  lookup table
A =zeros(1,nHospital);
B=zeros(1,nHospital);

% Assigning value to look up tables
nCount  = 0;
for i = 1:nHospital
    nCount = nCount+1;
    A(i)= nLookUpCounter+nCount;
    B(i)= nCount;
end
nLookUpCounter= nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B] =...
    BuildNurseLookup(nHospital,nLookUpCounter)
%This is function build the look up table
%and reverse Look up table

%==========Output  Variable======
% Matrix A for tracking the number
%(integer of nurse  is assigned  to hospital i

%Matrix B for tracking DV sol is related to nurse from hospital i

% nLookUp Counter to track how many DV Specialist C use

%==========input variable========
%nHospital =  number of hospital to be assigned
%nSpecialistC = number of Specialist C available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended

%Initialize Look up table and reverse  lookup table

A =zeros(1,nHospital);
B=zeros(1,nHospital);

% Assigning value to look up tables

nCount  = 0;
for i = 1:nHospital
    nCount = nCount+1;
    A(i)= nLookUpCounter+nCount;
    B(i)= nCount;
```

```
end
nLookUpCounter= nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B] =...
    BuildWorkerSpecialistBLookup(nHospital,nSpecialistB,nLookUpCounter)
%This is function build the look up table and reverse Look up table
%==========Output  Variable=======

% Matrix A for tracking whether specialist Bj works in hospital i
%Bj is an inidvidual who's career is specialist B

%Matrix B for tracking binary DV variable is equal to which
%  speacialist Bj work in which and hospital i (row)

% nLookUp Counter to track how many DV Specialist B use

%==========input variable===============
%nHospital =  number of hospital to be assigned
%nSpecialistB = number of Specialist B available to be assigned
%nLookUpCounter = Keep track of where the previous counter ended
%Initialize Look up table
    A= zeros(nHospital,nSpecialistB);
    B= zeros(nHospital*nSpecialistB,2);
%Assigning value
nCount = 0;
    for i = 1:nSpecialistB
        for j = 1:nHospital
            nCount = nCount+1;
            A(j,i) = nLookUpCounter+nCount;
            B(nCount,:) = [j,i];
        end
    end
%Update nLookUp Cpunter
    nLookUpCounter = nLookUpCounter+nCount;
end


function[A,nLookUpCounter,B] =...
    BuildPenaltyLookup(nSpecialistB,nLookUpCounter)

%Initialize Look up table and reverse  lookup table
A =zeros(1,nSpecialistB);
B=zeros(1,nSpecialistB);

% Assigning value to look up tables
```

```
nCount  = 0;
for i = 1:nSpecialistB
    nCount = nCount+1;
    A(i)= nLookUpCounter+nCount;
    B(i)= nCount;
end
nLookUpCounter= nLookUpCounter+nCount;
end



function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
    Constraint1(nHospital,nDV,nSpecialistA,...
    SpecialistALookUp,...
    aafALEQ,afbRHSLEQ,nLEQCounter)

    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        NewConstraint=zeros(1,nDV);
        for j = 1:nSpecialistA
            NewConstraint(SpecialistALookUp(i,j)) = -j ;
        end
        NewConstraint;
        aafALEQ(nLEQCounter,:) =NewConstraint;
        afbRHSLEQ(nLEQCounter)=-1;
    end
end


function [aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint2(nHospital,nSpecialistB,...
       SpecialistBLookUp,aafALEQ,afbRHSLEQ,nLEQCounter)
    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
        for j = 1:nSpecialistB
            aafALEQ (nLEQCounter,SpecialistBLookUp(i,j)) = 1;
            afbRHSLEQ (nLEQCounter) = 1;
        end
    end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint3(nHospital,nSpecialistC,...
       SpecialistCLookUp,aafALEQ,afbRHSLEQ,nLEQCounter)
    for i = 1:nHospital
        nLEQCounter = nLEQCounter+1;
```

```
        for j = 1:nSpecialistC
            aafALEQ (nLEQCounter,SpecialistCLookUp(i,j)) = 1;
            afbRHSLEQ (nLEQCounter) = 1;
        end
    end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter] =...
       Constraint4(nHospital,nDV,nSpecialistC,...
       SpecialistCLookUp,nSpecialistB,SpecialistBLookUp...
       ,aafALEQ,afbRHSLEQ,nLEQCounter)

       for i = 1:nHospital
           nLEQCounter = nLEQCounter+1;
           NewConstraint = zeros(1,nDV);
           for j =1:nSpecialistB
               NewConstraint(SpecialistBLookUp(i,j)) = -j ;
           end
           for j = 1:nSpecialistC
               NewConstraint(SpecialistCLookUp(i,j)) = -j ;
           end
           %NewConstraint
           aafALEQ(nLEQCounter,:) = NewConstraint;
           afbRHSLEQ(nLEQCounter) = -1;
       end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint5(nHospital,nDV,nSpecialistD,...
       SpecialistDLookUp,...
       aafALEQ,afbRHSLEQ,nLEQCounter)

   for i = 1:nHospital
       nLEQCounter = nLEQCounter+1;
       NewConstraint = zeros(1,nDV);
       NewConstraint(SpecialistDLookUp(i)) = 1;
       aafALEQ(nLEQCounter,:) = NewConstraint;
       afbRHSLEQ(nLEQCounter) = 1;
   end
end


function[aafALEQ,afbRHSLEQ,nLEQCounter]=...
       Constraint6(nHospital,nDV,nNurse,NurseLookUp,...
       aafALEQ,afbRHSLEQ,nLEQCounter)
```

```
   for i = 1:nHospital
       nLEQCounter = nLEQCounter+1;
       NewConstraint = zeros(1,nDV);
       NewConstraint(NurseLookUp(i)) = -1;
       aafALEQ(nLEQCounter,:) = NewConstraint;
       afbRHSLEQ(nLEQCounter) = -20;
   end

end


function[aafAEQ,afbRHSEQ,nEQCounter]=...
   Constraint7(nHospital,nDV,nSpecialistA,SpecialistALookUp,...
   aafAEQ,afbRHSEQ,nEQCounter)
        for i = 1:nHospital
            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
            for j = 1:nSpecialistA
                NewConstraint(SpecialistALookUp(i,j))=1;
            end
            %NewConstraint;
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =1;
        end

end


function[aafAEQ,afbRHSEQ,nEQCounter]=...
       Constraint8(nHospital,nDV,nSpecialistA,...
       SpecialistALookUp,aafAEQ,afbRHSEQ,nEQCounter)
                nEQCounter = nEQCounter+1;
                NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                     for j = 1:nSpecialistA
                        NewConstraint(SpecialistALookUp(i,j))=j;
                     end
                end
            %NewConstraint;
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =8;
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint9(nHospital,nDV,nSpecialistC,...
```

```
       SpecialistCLookUp,nSpecialistB,SpecialistBLookUp,...
       aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                     for j = 1:nSpecialistB
                        NewConstraint(SpecialistBLookUp(i,j))=j;
                     end
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =3;
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint10(nHospital,nDV,nSpecialistC,...
       SpecialistCLookUp,nSpecialistB,SpecialistBLookUp,...
       aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                     for j = 1:nSpecialistC
                        NewConstraint(SpecialistCLookUp(i,j))=j;
                     end
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =4;
end


function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint11(nHospital,nDV,nSpecialistD,...
       SpecialistDLookUp,aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                        NewConstraint(SpecialistDLookUp(i))=1;
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =3;
end
```

```
function[aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint12(nHospital,nDV,nNurse,...
       NurseLookUp,aafAEQ,afbRHSEQ,nEQCounter)

            nEQCounter = nEQCounter+1;
            NewConstraint = zeros(1,nDV);
                for i = 1:nHospital
                        NewConstraint(NurseLookUp(i))=1;
                end
            %NewConstraint
            aafAEQ(nEQCounter,:)= NewConstraint;
            afbRHSEQ(nEQCounter) =250;
end


function[aafAEQ,afbRHSEQ,nEQCounter]=...
          Constraint13(nHospital,nDV,nWorkerSolDV,...
          nSpecialistB,WorkerBLookup,...
          SpecialistBLookUp,aafAEQ,afbRHSEQ,nEQCounter)

        for i = 1:nHospital
            nEQCounter = nEQCounter+1;
            NewConstraint=zeros(1,nDV);
            for j =1:nSpecialistB
                NewConstraint(1,WorkerBLookup(i,j))=1;
                NewConstraint(1,SpecialistBLookUp(i,j))=-j;
            end
            %NewConstraint
            aafAEQ(nEQCounter,:)=NewConstraint;
            afbRHSEQ(nEQCounter)=0;
        end
end


function[aafAEQ,afbRHSEQ,nEQCounter]=...
          Constraint14(nHospital,nDV,nWorkerSolDV,...
          nSpecialistB,WorkerBLookup,SpecialistBLookUp,...
          aafAEQ,afbRHSEQ,nEQCounter)

        nCount = 0 ;
        for j = 1:nSpecialistB
            NewConstraint = zeros(1,nDV);
            nCount= nCount+1;
            for i = 1:nHospital
                NewConstraint(1,WorkerBLookup(i,j) )=1;
```

```
            end
             NewConstraint(1,nWorkerSolDV+nCount)=-1;
             %NewConstraint
             aafAEQ(nEQCounter+nCount,:) = NewConstraint;
             afbRHSEQ(nEQCounter+nCount) = 1;

        end
            nEQCounter = nEQCounter+nCount;
end


function[SpecialistALookUp,...
    SpecialistAReverseLookUp,nSpecialistASolDV,...
            SpecialistBLookUp,...
            SpecialistBReverseLookUp,nSpecialistBSolDV,...
            SpecialistCLookUp,....
            SpecialistCReverseLookUp,nSpecialistCSolDV,...
            SpecialistDLookUp,...
            SpecialistDReverseLookUp,nSpecialistDSolDV,...
            NurseLookUp,NurseReverseLookUp,nNurseSolDV,...
            WorkerBLookup,...
            nWorkerSolDV,WorkerBReverseLookUp,...
            PenaltyLookUp,nLookUpCounter,...
            PenaltyReverseLookUp] =...
        buildLookUp(nHospital,nSpecialistA,...
        nSpecialistB,nSpecialistC,nSpecialistD,nNurse )

    %Initialize DV counter
        nLookUpCounter = 0 ;
    %Create Look up and Reverse Look for Specialist A
        [SpecialistALookUp,nLookUpCounter,SpecialistAReverseLookUp] = ...
            BuildSpecialistALookup(nHospital,nSpecialistA,nLookUpCounter);
    %Track where Specialist A DV ends
        nSpecialistASolDV = nLookUpCounter;
    %Create Lookup and reverse Lookup for Specialist B
        [SpecialistBLookUp,nLookUpCounter,SpecialistBReverseLookUp] = ...
        BuildSpecialistBLookup(nHospital,nSpecialistB,nLookUpCounter);
    %Track where Specialist B DV ends
        nSpecialistBSolDV = nLookUpCounter;
    %Build LookUp and Reverse Look up table for Specialist C
        [SpecialistCLookUp,nLookUpCounter,SpecialistCReverseLookUp] = ...
            BuildSpecialistCLookup(nHospital,nSpecialistC,nLookUpCounter);
    %Track Where Specialist C DV ends
        nSpecialistCSolDV = nLookUpCounter;
    %Build LookUp and Reverse Look up table for Specialist D
        [SpecialistDLookUp,nLookUpCounter,SpecialistDReverseLookUp] = ...
            BuildSpecialistDLookup(nHospital,nLookUpCounter);
```

```
    %Track where Specialist D DV  ends
        nSpecialistDSolDV = nLookUpCounter;
    %Build LookUp and Reverse Look up table for Nurse Practitioner
        [NurseLookUp,nLookUpCounter,NurseReverseLookUp] = ...
            BuildNurseLookup(nHospital,nLookUpCounter);
    %Track where Nurse DV  ends
        nNurseSolDV = nLookUpCounter;
    %Build Specialist B worker tracker
    [WorkerBLookup,nLookUpCounter,WorkerBReverseLookUp]=...
        BuildWorkerSpecialistBLookup(nHospital,nSpecialistB,nLookUpCounter);
    %Track where Worker tracker DV end
    nWorkerSolDV = nLookUpCounter;
    %Build Penalty LookUp table
    [PenaltyLookUp,nLookUpCounter,PenaltyReverseLookUp] = ...
        BuildPenaltyLookup(nSpecialistB,nLookUpCounter);
    %Track where Penalty DV end
    nPenaltySolDV = nLookUpCounter;

end


function[aafALEQ,afbRHSLEQ,aafAEQ,...
     afbRHSEQ,nLEQCounter,nEQCounter]=...
    buildConstraint(nHospital,nDV,aafSpecialistBenefit,...
        nSpecialistA,nSpecialistB,nSpecialistC,...
        nSpecialistD,nNurse,...
        SpecialistALookUp,...
        SpecialistAReverseLookUp,nSpecialistASolDV,...
        SpecialistBLookUp,...
        SpecialistBReverseLookUp,nSpecialistBSolDV,...
        SpecialistCLookUp,...
        SpecialistCReverseLookUp,nSpecialistCSolDV,...
        SpecialistDLookUp,...
        SpecialistDReverseLookUp,nSpecialistDSolDV,...
        NurseLookUp,NurseReverseLookUp, nNurseSolDV,...
        WorkerBLookup,nWorkerSolDV,WorkerBReverseLookUp,...
        aafALEQ,afbRHSLEQ,aafAEQ,afbRHSEQ,...
        nLEQCounter,nEQCounter)
    %-------------------------------------------------------------
    %            Less or equal Constraints
    %-------------------------------------------------------------

    % can only pick berween 1,2,or 3 for specialist B in each hospital
        [aafALEQ,afbRHSLEQ,nLEQCounter]=...
           Constraint2(nHospital,nSpecialistB,...
           SpecialistBLookUp,aafALEQ,afbRHSLEQ,nLEQCounter);
    %can only pick between 1,2,3,4 for specialist C in each hospital
```

```
        [aafALEQ,afbRHSLEQ,nLEQCounter]=...
           Constraint3(nHospital,nSpecialistC,...
           SpecialistCLookUp,aafALEQ,afbRHSLEQ,nLEQCounter);
    %atleast one specialist B or C in each hospital
       [aafALEQ,afbRHSLEQ,nLEQCounter]=...
           Constraint4(nHospital,nDV,nSpecialistC,...
           SpecialistCLookUp,nSpecialistB,SpecialistBLookUp,...
           aafALEQ,afbRHSLEQ,nLEQCounter);


    %-------------------------------------------------------------
    %              Equality Constraints
    %-------------------------------------------------------------


%can either pick 1,2 ,3 ,..8 for Specalist A in each hospital
    [aafAEQ,afbRHSEQ,nEQCounter]=...
       Constraint7(nHospital,nDV,nSpecialistA,...
       SpecialistALookUp,aafAEQ,afbRHSEQ,nEQCounter);

  % can only assigned 8 Specialist A in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter]=...
       Constraint8(nHospital,nDV,nSpecialistA,...
       SpecialistALookUp,aafAEQ,afbRHSEQ,nEQCounter);

    % can only assigned 4 specialist C in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint10(nHospital,nDV,nSpecialistC,...
       SpecialistCLookUp,nSpecialistB,SpecialistBLookUp...
       ,aafAEQ,afbRHSEQ,nEQCounter);

   %Can only assigned 3 Specialist D in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint11(nHospital,nDV,nSpecialistD,...
       SpecialistDLookUp,aafAEQ,afbRHSEQ,nEQCounter);

   % Can only assigned 250 Nurses in 5 hospital
   [aafAEQ,afbRHSEQ,nEQCounter] =...
       Constraint12(nHospital,nDV,nNurse,...
       NurseLookUp,aafAEQ,afbRHSEQ,nEQCounter);

    %Relate WorkerB with binary variable # n
    %number Specialist Type B assigned to hospital i
    [aafAEQ,afbRHSEQ,nEQCounter]=...
      Constraint13(nHospital,nDV,nWorkerSolDV,nSpecialistB,...
      WorkerBLookup,SpecialistBLookUp,...
      aafAEQ,afbRHSEQ,nEQCounter);
```

```
    %Constraint that track penalty
    [aafAEQ,afbRHSEQ,nEQCounter]=...
      Constraint14(nHospital,nDV,nWorkerSolDV,nSpecialistB,...
      WorkerBLookup,SpecialistBLookUp,...
      aafAEQ,afbRHSEQ,nEQCounter);

end


function[afUB] = createUpperBound(afUB,...
     nSpecialistDSolDV,nNurseSolDV,nWorkerSolDV,nDV)
         %upper bound for Nurse
            for i=nSpecialistDSolDV+1:nNurseSolDV
               afUB(i) =50;
            end
            for i=nWorkerSolDV+1:nDV
                afUB(i) = 5;
            end
end


function[afLB] = createLowerBound(afLB,...
    nSpecialistDSolDV,nNurseSolDV,nDV)
    %Lower bound for nurses
         for i=nSpecialistDSolDV+1:nNurseSolDV
           afLB(i) =20;
        end
end

function[C] = buildObjectives(...
            C,nDV,nPenalty,aafSpecialistBenefit,...
            SpecialistALookUp,...
            SpecialistAReverseLookUp,nSpecialistASolDV,...
            SpecialistBLookUp,...
            SpecialistBReverseLookUp,nSpecialistBSolDV,...
            SpecialistCLookUp,...
            SpecialistCReverseLookUp,nSpecialistCSolDV,...
            SpecialistDLookUp,...
            SpecialistDReverseLookUp,nSpecialistDSolDV,...
            WorkerBLookup,nWorkerSolDV,WorkerBReverseLookUp,...
            NurseLookUp,NurseReverseLookUp, nNurseSolDV)

        NewConstraint= zeros(1,nDV);
       for i=1:nSpecialistASolDV
           NewConstraint(i) =...
               aafSpecialistBenefit(SpecialistAReverseLookUp(i,2),1);
```

```
       end

        for i=nSpecialistASolDV+1:nSpecialistBSolDV
           NewConstraint(i) =...
               aafSpecialistBenefit(...
               SpecialistBReverseLookUp(i-nSpecialistASolDV,2),2);
        end

        for i=nSpecialistBSolDV+1:nSpecialistCSolDV
           NewConstraint(i) =...
               aafSpecialistBenefit(...
               SpecialistCReverseLookUp(i-nSpecialistBSolDV,2),3);
        end
        for i=nSpecialistCSolDV+1:nSpecialistDSolDV
           NewConstraint(i) =2;
        end
        for i=nSpecialistDSolDV+1:nNurseSolDV
           NewConstraint(i) =1;
        end
        for i = nWorkerSolDV+1:nDV
            NewConstraint(i) = -nPenalty;
        end
        C(1,:)=NewConstraint;
end

function[aafAns] = buildAnswer(nDV,nHospital,...
                    SpecialistALookUp,nSpecialistA,...
                    SpecialistBLookUp,nSpecialistB,...
                     SpecialistCLookUp,nSpecialistC,...
                     SpecialistDLookUp,nSpecialistD,...
                     NurseLookUp, nNurse,...
                     aafAns)
        nAnsCount = 0 ;

    %Specialist A
        for  i = 1:nHospital
            NewConstraint = zeros(1,nDV);
            nAnsCount = nAnsCount +1 ;
            for j = 1:nSpecialistA
                NewConstraint(SpecialistALookUp(i,j)) = j ;
            end
            aafAns(nAnsCount,:) = NewConstraint;
        end

    %Specialist B
         for  i = 1:nHospital
            NewConstraint = zeros(1,nDV);
```

```
            nAnsCount = nAnsCount +1 ;
            for j = 1:nSpecialistB
                NewConstraint(SpecialistBLookUp(i,j)) = j ;
            end
            aafAns(nAnsCount,:) = NewConstraint;
         end

    %Specialist C
         for  i = 1:nHospital
            NewConstraint = zeros(1,nDV);
            nAnsCount = nAnsCount +1 ;
            for j = 1:nSpecialistC
                NewConstraint(SpecialistCLookUp(i,j)) = j ;
            end
            aafAns(nAnsCount,:) = NewConstraint;
         end

    %Specialist D
        for i = 1 :nHospital
            nAnsCount=nAnsCount+1;
            aafAns(nAnsCount,SpecialistDLookUp(i)) = 1;
        end

     %NursePractitioner
        for i = 1 :nHospital
            nAnsCount=nAnsCount+1;
            %nurse in   unit of tens of people
            aafAns(nAnsCount,NurseLookUp(i)) = 1/10;
        end
    end

function[]=makeBarPlot(nHospital,nTypeWorker,afAllocation)
% Preparing data for bar plot
    aafBarMatrix = zeros(nHospital,nTypeWorker);
    nCounter = 0 ;
     for i = 1:nHospital
         for j = 1:nTypeWorker
             nCounter = nCounter+1;
             aafBarMatrix(j,i) = afAllocation(nCounter);
         end
     end
     aafBarMatrix;

%-----------------------------------------------------------------
%Creating Bar Plot
%-----------------------------------------------------------------
```

```
    figure()
    b = bar(aafBarMatrix,'stacked');

    b(1).FaceColor = [0.3,0.3,0.3];
    b(2).FaceColor = [0.4,0.4,0.4];
    b(3).FaceColor = [0.5,0.5,0.5];
    b(4).FaceColor = [0.6,0.6,0.6];
    b(5).FaceColor = [0.7,0.7,0.7];
    xlabel ('Hospital')
    cTitle = ['Number of Workers Assigned in Each Hospital'];
    title(cTitle);
    xlim([0,6]);
    ylim([0,30]);

    legend('Specialist A','Specialist B', 'Specialist C', ...
        'Specialist D','Nurse (in tens of people)')
 end
```

```
LP:                Optimal objective value is -287.000000.


Optimal solution found.

Intlinprog stopped at the root node because the objective value is within a gap
tolerance of the optimal value, options.AbsoluteGapTolerance = 0 (the default
value). The intcon variables are integer within tolerance,
options.IntegerTolerance = 1e-05 (the default value).

Hospital 1 have 2 Specialist Type A
Hospital 2 have 2 Specialist Type A
Hospital 3 have 2 Specialist Type A
Hospital 4 have 1 Specialist Type A
Hospital 5 have 1 Specialist Type A
Hospital 1 have 1.000000e+00 Specialist Type B
Hospital 2 have 1.000000e+00 Specialist Type B
Hospital 3 have 1.000000e+00 Specialist Type B
Hospital 4 have 1.000000e+00 Specialist Type B
Hospital 5 have 0 Specialist Type B
Hospital 1 have 0 Specialist Type C
Hospital 2 have 0 Specialist Type C
Hospital 3 have 0 Specialist Type C
Hospital 4 have 0 Specialist Type C
Hospital 5 have 4 Specialist Type C
Hospital 1 have 1 Specialist Type D
Hospital 2 have 1 Specialist Type D
Hospital 3 have 1 Specialist Type D
```

```
Hospital 4 have 0 Specialist Type D
Hospital 5 have 0 Specialist Type D
Hospital 1 have 50 Nurse
Hospital 2 have 50 Nurse
Hospital 3 have 50 Nurse
Hospital 4 have 50 Nurse
Hospital 5 have 50 Nurse
Specialist type B number 1 work in hospital 2
Specialist type B number 1 work in hospital 3
Specialist type B number 2 work in hospital 4
Specialist type B number 3 work in hospital 1
penalty from worker  1 is 1
penalty from worker  2 is 0
penalty from worker  3 is 0
```

References

[1] F Hiller and G Lieberman, *Introduction to Operations Research*. McGraw-Hill Education, 2014.

[2] J Cannons, *Mat 3810 lecture notes*, 2020.